Agentic Workflow Platform: Technical Implementation & Architecture Specification

1. Executive Summary & Architectural Overview

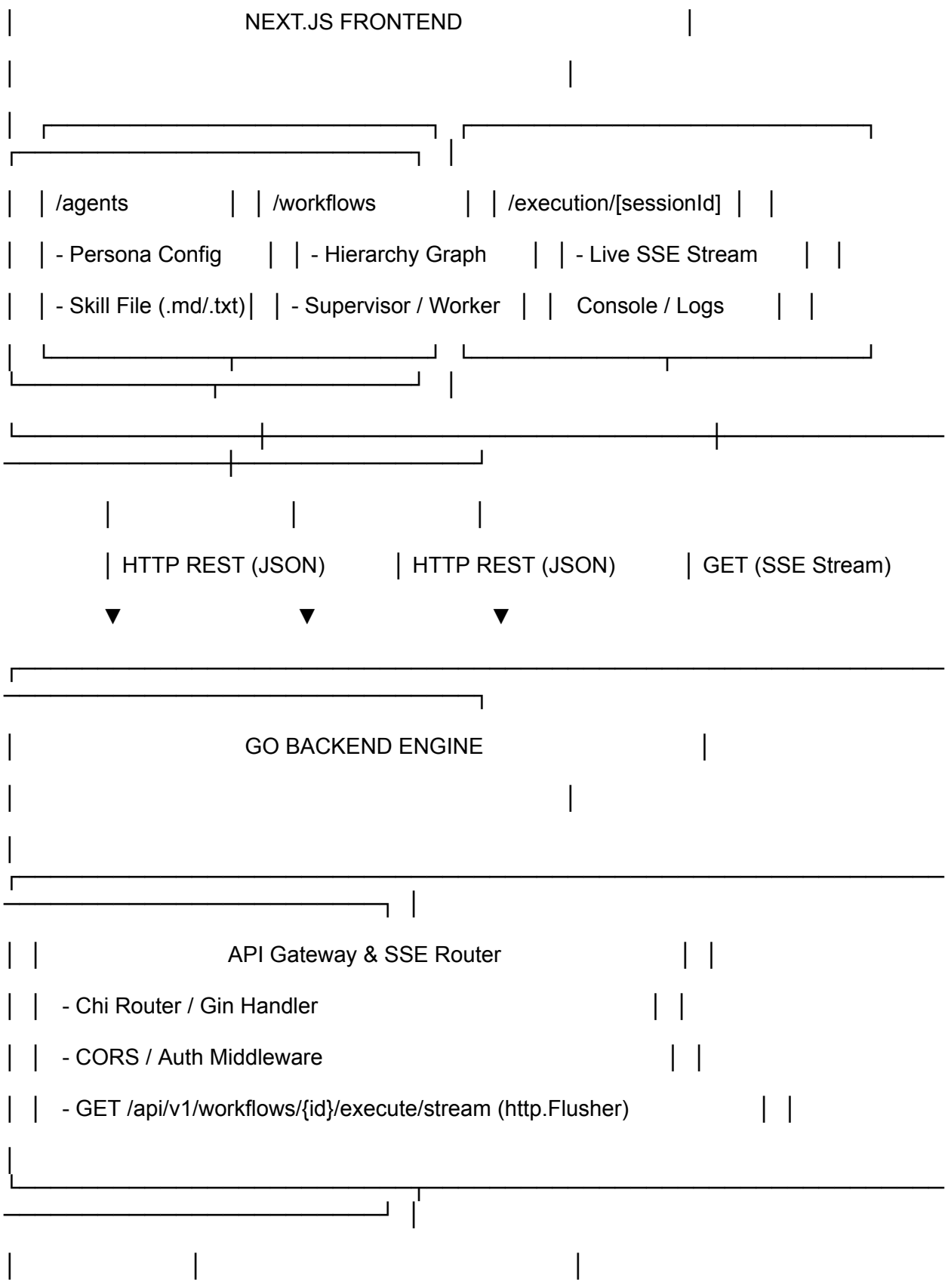
This document details the architectural design and technical specification for a multi-agent orchestration platform. The system enables end-users to dynamically build, configure, and execute hierarchical AI agent workflows. Key capabilities include customizable agent personas, multi-skill attachment (Markdown and raw text file ingestion), standardized function calling via Model Context Protocol (MCP), persistent execution session logging, and real-time streaming of agent cognition via Server-Sent Events (SSE).

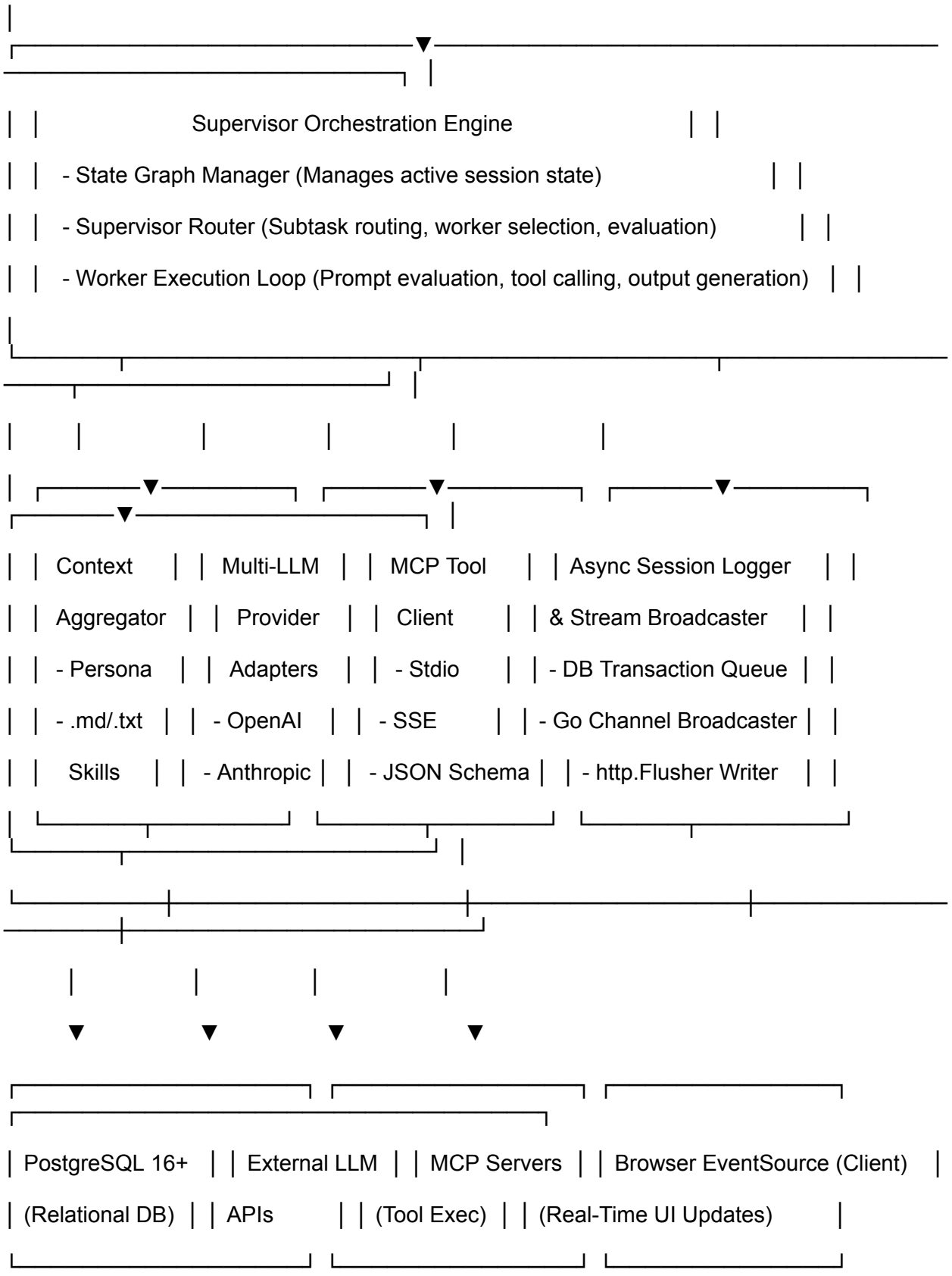
1.1 High-Level Tech Stack

Layer	Technology Choice	Primary Responsibilities
Frontend	Next.js 14+ (App Router, TypeScript, Tailwind CSS)	Agent & skill management UI, visual workflow builder, real-time SSE stream renderers.
Backend Engine	Go (Golang 1.22+)	High-concurrency state execution, supervisor-worker orchestration, MCP tool client, SSE HTTP streaming.
Database & Persistence	PostgreSQL 16+	Relational storage for agents, persona profiles, skill chunks, workflow graphs, and session audit logs.
Tool Protocol	Model Context Protocol (MCP)	Standardized interface for local and remote function calling and external tool invocation.

2. System Domain Model & Database Schema Design

The system state is persisted in PostgreSQL to ensure strict relational integrity, execution repeatability, and granular audit trails of agent reasoning loops.


NEXT.JS FRONTEND
/agents
- Persona Config
- Skill File (.md/.txt)
/workflows
- Hierarchy Graph
- Supervisor / Worker
/execution/[sessionId]
- Live SSE Stream
Console / Logs
HTTP REST (JSON)
HTTP REST (JSON)
GET (SSE Stream)
GO BACKEND ENGINE
API Gateway & SSE Router
- Chi Router / Gin Handler
- CORS / Auth Middleware
- GET /api/v1/workflows/{id}/execute/stream (http.Flusher)


Supervisor Orchestration Engine
- State Graph Manager (Manages active session state)
- Supervisor Router (Subtask routing, worker selection, evaluation)
- Worker Execution Loop (Prompt evaluation, tool calling, output generation)
Context Aggregator
- Persona
- .md/.txt
Skills
Multi-LLM Provider
Adapters
- OpenAI
- Anthropic
- JSON Schema
MCP Tool Client
- Stdio
- SSE
Async Session Logger & Stream Broadcaster
- DB Transaction Queue
- Go Channel Broadcaster
- http.Flusher Writer
PostgreSQL 16+
(Relational DB)
External LLM
APIs
MCP Servers
(Tool Exec)
Browser EventSource (Client)
(Real-Time UI Updates)

└──┘

```
backend/
├── cmd/
│   └── server/
│       └── main.go              # Entrypoint: initializes DB, MCP, HTTP routers
├── internal/
│   ├── config/                  # Environment configuration & secrets
│   ├── domain/                  # Entities: Agent, Skill, Workflow, SessionLog
│   ├── db/                      # PostgreSQL queries, migrations, connections
│   ├── llm/                     # Provider Abstraction Interface
│   │   ├── provider.go          # LLMProvider interface definition
│   │   ├── openai.go            # OpenAI API client adapter
│   │   ├── anthropic.go         # Anthropic Claude API adapter
│   │   └── gemini.go            # Google Gemini API adapter
│   ├── mcp/                     # Model Context Protocol Engine
│   │   ├── client.go            # MCP Client manager (Stdio/SSE transports)
│   │   └── translator.go        # Converts MCP tool signatures to LLM JSON Schema
│   ├── orchestrator/            # Multi-Agent Workflow Engine
│   │   ├── supervisor.go        # Supervisor routing logic & subtask dispatcher
│   │   ├── worker.go            # Worker execution ReAct loop
│   │   ├── context.go           # Context Aggregator (Persona + Skills injection)
│   │   └── state.go             # In-memory execution state graph
│   ├── sse/                     # Real-time Streaming Engine
│   │   ├── broadcaster.go       # Go channels for thread-safe event streaming
```

```
│   │   └── handler.go            # HTTP Handler with http.Flusher
│   └── api/                      # HTTP REST Handlers
│       ├── agent_handler.go      # CRUD for agents & skill attachments
│       ├── workflow_handler.go   # CRUD for workflow hierarchies
│       └── execution_handler.go  # Trigger workflow & SSE streaming endpoint
```

2.1 Schema Definition (PostgreSQL DDL)

```
-- 1. Agents Table
CREATE TABLE agents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    persona TEXT NOT NULL,
    model_provider VARCHAR(100) NOT NULL DEFAULT 'openai',
    model_name VARCHAR(100) NOT NULL DEFAULT 'gpt-4o',
    temperature NUMERIC(3, 2) DEFAULT 0.20,
    role_type VARCHAR(50) NOT NULL CHECK (role_type IN ('supervisor',
'worker', 'evaluator')),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Skills Table (Markdown / Text files attached to agents)
CREATE TABLE skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    file_type VARCHAR(20) NOT NULL CHECK (file_type IN ('markdown',
'text')),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Agent Skills Junction (Multiple skills per agent)
CREATE TABLE agent_skills (
    agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    PRIMARY KEY (agent_id, skill_id)
);

-- 4. MCP Tools Registry
CREATE TABLE mcp_tools (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    name VARCHAR(255) UNIQUE NOT NULL,
    description TEXT NOT NULL,
    server_url TEXT NOT NULL,
    input_schema JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 5. Agent Tools Junction
CREATE TABLE agent_mcp_tools (
    agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,
    mcp_tool_id UUID REFERENCES mcp_tools(id) ON DELETE CASCADE,
    PRIMARY KEY (agent_id, mcp_tool_id)
);

-- 6. Hierarchical Workflows
CREATE TABLE workflows (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    supervisor_agent_id UUID REFERENCES agents(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 7. Workflow Nodes & Hierarchy Definition
CREATE TABLE workflow_nodes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID REFERENCES workflows(id) ON DELETE CASCADE,
    parent_node_id UUID REFERENCES workflow_nodes(id),
    agent_id UUID REFERENCES agents(id),
    execution_order INT NOT NULL,
    routing_condition TEXT
);

-- 8. Execution Sessions & Execution Trace Logs
CREATE TABLE execution_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID REFERENCES workflows(id),
    status VARCHAR(50) NOT NULL DEFAULT 'RUNNING',
    input_query TEXT NOT NULL,
    final_output TEXT,
    started_at TIMESTAMPTZ DEFAULT NOW(),
    completed_at TIMESTAMPTZ
);

CREATE TABLE session_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID REFERENCES execution_sessions(id) ON DELETE
CASCADE,
```

```
    agent_id UUID REFERENCES agents(id),
    step_number INT NOT NULL,
    log_type VARCHAR(50) NOT NULL, -- THOUGHT, TOOL_CALL, TOOL_RESULT,
DELEGATION, FINAL_RESPONSE
    content JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

3. Go Backend Architecture & Engine Design

The Go backend acts as the core orchestration runtime. It loads agent personas, aggregates skill context, binds MCP tools, and manages concurrent step-by-step supervisor routing.

3.1 Engine Modules

- **Context Aggregator:** Combines base persona system prompts with attached Markdown/text skill contents into a unified agent context window.
- **Supervisor Router:** Evaluates subtask state, determines delegate worker agents, and collects intermediate worker outputs.
- **MCP Client Manager:** Connects to MCP servers over SSE or Stdio transports, registers tools into standard JSON Schema format, and executes remote tool calls.
- **SSE Event Streamer:** Streams real-time JSON event packets to HTTP clients while concurrently writing audit logs to PostgreSQL.

3.2 Server-Sent Events (SSE) Protocol Implementation in Go

The Go backend exposes an SSE HTTP stream handler. Below is a production-grade implementation pattern using standard net/http and Go channels:

```
package sse

import (
    "encoding/json"
    "fmt"
    "net/http"
)

type EventType string

const (
    EventAgentThought     EventType = "AGENT_THOUGHT"
    EventDelegation       EventType = "AGENT_DELEGATION"
    EventToolCall         EventType = "TOOL_CALL"
    EventToolResult       EventType = "TOOL_RESULT"
    EventTokenStream      EventType = "TOKEN_STREAM"
    EventWorkflowComplete EventType = "WORKFLOW_COMPLETE"
    EventError            EventType = "ERROR"
```

```
)

type StreamMessage struct {
    Event     EventType   `json:"event"`
    SessionID string      `json:"session_id"`
    AgentName string      `json:"agent_name,omitempty"`
    Step      int         `json:"step"`
    Payload   interface{} `json:"payload"`
}

func HandleSSEStream(w http.ResponseWriter, r *http.Request, eventChan
<-chan StreamMessage) {
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")
    w.Header().Set("X-Accel-Buffering", "no")

    flusher, ok := w.(http.Flusher)
    if !ok {
        http.Error(w, "Streaming unsupported",
http.StatusInternalServerError)
        return
    }

    for {
        select {
        case <-r.Context().Done():
            return
        case msg, open := <-eventChan:
            if !open {
                fmt.Fprintf(w, "event: close\ndata: {}\n\n")
                flusher.Flush()
                return
            }
            data, _ := json.Marshal(msg)
            fmt.Fprintf(w, "event: %s\ndata: %s\n\n", msg.Event, data)
            flusher.Flush()
        }
    }
}
```

4. Next.js Frontend Integration & Real-Time SSE Consumer

The Next.js client renders dynamic management interfaces and streams reasoning progress from the Go backend using Server-Sent Events.

```
frontend/
├── app/
│   ├── layout.tsx              # Root layout with navbar & providers
│   ├── page.tsx                # Dashboard home
│   ├── agents/
│   │   ├── page.tsx            # Agent directory & grid view
│   │   └── create/
│   │       └── page.tsx        # Form: Persona, Model selection, Skill uploader
│   ├── workflows/
│   │   ├── page.tsx            # Saved workflows list
│   │   └── builder/
│   │       └── page.tsx        # Visual Node hierarchy builder (Supervisor -> Workers)
│   └── sessions/
│       └── [id]/
│           └── page.tsx        # Live streaming execution monitor & trace viewer
├── components/
│   ├── agents/
│   │   ├── AgentCard.tsx       # Displays persona, skills count, model
│   │   └── SkillUploader.tsx   # Drag-and-drop file parser (.md / .txt)
│   ├── workflows/
│   │   └── NodeTree.tsx        # Renders hierarchical relationship tree
│   └── streaming/
│       ├── ThoughtConsole.tsx  # Renders streaming thinking tokens & reasoning steps
│       ├── ToolCallBadge.tsx   # Expandable JSON inspector for MCP tool calls
```

```
│      └── ExecutionTimeline.tsx   # Step-by-step visual timeline of agent handoffs
├── hooks/
│   ├── useWorkflowExecution.ts     # Custom hook for SSE stream management
│   └── useAgents.ts              # TanStack / SWR fetchers for REST APIs
└── lib/
    ├── api.ts                  # Axios/Fetch client for REST calls
    └── types.ts                # TypeScript interface definitions
```

4.1 SSE Consumption Hook (React/TypeScript)

```
import { useState } from 'react';

export interface SSELogEvent {
  event: 'AGENT_THOUGHT' | 'AGENT_DELEGATION' | 'TOOL_CALL' |
'TOOL_RESULT' | 'TOKEN_STREAM' | 'WORKFLOW_COMPLETE';
  session_id: string;
  agent_name?: string;
  step: number;
  payload: any;
}

export function useWorkflowExecution(workflowId: string, query:
string) {
  const [logs, setLogs] = useState<SSELogEvent[]>([]);
  const [status, setStatus] = useState<'idle' | 'running' |
'completed' | 'error'>('idle');

  const startExecution = () => {
    setLogs([]);
    setStatus('running');

    const encodedQuery = encodeURIComponent(query);
    const eventSource = new
EventSource(`/api/v1/workflows/${workflowId}/execute/stream?query=${en
codedQuery}`);

    eventSource.onmessage = (event) => {
      const parsed: SSELogEvent = JSON.parse(event.data);
      setLogs((prev) => [...prev, parsed]);
```

```
      if (parsed.event === 'WORKFLOW_COMPLETE') {
        setStatus('completed');
        eventSource.close();
      }
    };

    eventSource.onerror = (err) => {
      console.error("SSE Connection Error:", err);
      setStatus('error');
      eventSource.close();
    };
  };

  return { logs, status, startExecution };
}
```

5. API Endpoint Specifications

Method	Endpoint	Description	Format
POST	/api/v1/agents	Create new agent profile with persona & model settings.	JSON
POST	/api/v1/skills	Upload Markdown or raw text skill document.	Multipart / JSON
POST	/api/v1/agents/{id}/skills	Attach one or multiple skill IDs to an agent.	JSON
POST	/api/v1/mcp/tools	Register an MCP tool server and function definition.	JSON
POST	/api/v1/workflows	Define multi-agent hierarchy (Supervisor + Workers).	JSON
GET	/api/v1/workflows/{id}/execute/stream	Initiate workflow execution and stream real-time SSE events.	Text/Event-Stream
GET	/api/v1/sessions/{id}/logs	Retrieve persisted session execution logs from DB.	JSON

6. Phased Implementation Roadmap

1. **Phase 1: Domain Core & Agent Management** – Build Go CRUD services, database migrations, skill file parsing, and prompt context assembler.
2. **Phase 2: MCP Protocol Integration** – Implement Go client for MCP tool registration, JSON schema mapping, and execution runtime.

3. **Phase 3: Hierarchical Supervisor Engine** – Construct supervisor-worker execution loop in Go with state tracking.
4. **Phase 4: SSE & Database Session Persistence** – Wire up Go HTTP SSE flusher and asynchronous session logging to PostgreSQL.
5. **Phase 5: Next.js Visual UI** – Build workflow builder, skill uploader, and live stream reasoning log component.